

Algorithm

2023 年 9 月 26 日

目录

目录	2
1 dining	4
2 transform	7
3 power	13
4 expr	15
5 pow	20
6 number	21
7 decode	23
8 two-sum	26
9 candy	27
10 live	29
11 number	31
12 expr	33
13 point	35
14 fruit	38

目录	3
15 sort	40
16 solve	44
17 network	45
18 josephus	47
19 convention	48

1 dining

usaco-gold-2018-dining

The cows are heading back to the barn at the end of a long day, feeling both tired and hungry. The farm consists of N pastures ($2 \leq N \leq 50,000$), conveniently numbered $1 \cdots N$. The cows all want to travel to the barn in pasture N . Each of the other $N-1$ pastures contains a cow. Cows can move from pasture to pasture via a set of M undirected trails ($1 \leq M \leq 100,000$). The i th trail connects a pair of pastures a_i and b_i , and requires time t_i to traverse. Every cow can reach the barn through a sequence of trails.

Being hungry, the cows are interested in potentially stopping for food on their way home. Conveniently, K of the pastures contain tasty haybales ($1 \leq K \leq N$), with the i th such haybale having a yumminess value of y_i . Each cow is willing to stop at a single haybale along her trip to the barn, but only if the amount of time this adds to her path is at most the yumminess of the haybale she visits. Note that a cow only "officially" visits at most one haybale for dining purposes, although it is fine if her path takes her through other pastures containing haybales; she simply ignores these.

INPUT FORMAT (file dining.in):

The first line contains three space-separated integers N , M , and K . Each of the next M lines contains three integers a_i , b_i , and t_i , describing a trail between pastures a_i and b_i which takes t_i time to traverse (a_i and b_i are different from each-other, and t_i is a positive integer at most 10^4).

The next K lines each describe a haybale in terms of two integers: the index of its pasture, and its yumminess value (a positive integer at most 10^9). Multiple haybales can reside in the same pasture.

OUTPUT FORMAT (file dining.out):

The output should consist of $N-1$ lines. Line i contains the single integer 1 if the cow at pasture i can visit and dine on a haybale on the way to the barn, and 0 otherwise.

SAMPLE INPUT:

```
4 5 1
1 4 10
2 1 20
4 2 3
2 3 5
4 3 2
2 7
```

SAMPLE OUTPUT:

```
1
```

1

1

In this example, the cow in pasture 3 should stop for a meal, since her route would only increase by 6 (from 2 to 8), and this increase is at most the yumminess 7 of the haybale. The cow in pasture 2 should obviously eat the hay in pasture 2, since this causes no change in her optimal route. The cow in pasture 1 is an interesting case, as it may first appear that her optimal route (length 10) would increase too much to justify stopping for the hay. However, she actually does have a route that makes stopping at the hay beneficial: move to pasture 4, then to pasture 2 (eating the hay), then back to pasture 4.

解:

【条件分析】

1	N 个牧场	$2 \leq N \leq 50,000$
2	M 条无向路径	$1 \leq M \leq 100,000$
3	K 个牧场有干草	$1 \leq M \leq 10^5$
4	input	
4	output	

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define a array
```

```
const int maxN=5, BIG=1e9;
```

```
int n, m, k, y[maxN], ai, bi, ti, d1[maxN];
```

```
a<int, 2> d2[maxN];
```

```
vector<a<int, 2>> adj[maxN];
```

```
void dijkstra1() {
```

```
    priority_queue<a<int, 2>, vector<a<int, 2>>, greater<a<int, 2>>> pq;
```

```
    memset(d1, 0x7f, 4*n);
```

```
    d1[n-1]=0;
```

```
    pq.push({0, n-1});
```

```
    while(pq.size()) {
```

```
        a<int, 2> u=pq.top();
```

```
        pq.pop();
```

```
        if(u[0]>d1[u[1]])
```

```
            continue;
```

```

        for(a<int , 2> v : adj[u[1]]) {
            if(d1[v[1]]>u[0]+v[0]) {
                d1[v[1]]=u[0]+v[0];
                pq.push({d1[v[1]], v[1]});
            }
        }
    }
}

void dijkstra2() {
    priority_queue<a<int , 3>, vector<a<int , 3>>, greater<a<int ,
        3>>> pq;
    memset(d2, 0x7f, sizeof(d2[0])*n);
    d2[n-1]={BIG, n-1};
    pq.push({BIG, n-1, n-1});
    for(int i=0; i<n-1; ++i) {
        if(!y[i])
            continue;
        d2[i]={d1[i]+BIG-y[i], i};
        pq.push({d1[i]+BIG-y[i], i, i});
    }
    while(pq.size()) {
        a<int , 3> u=pq.top();
        pq.pop();
        if(a<int , 2>{u[0], u[1]}>d2[u[2]])
            continue;
        for(a<int , 2> v : adj[u[2]]) {
            if(d2[v[1]]>a<int , 2>{u[0]+v[0], u[1]}) {
                d2[v[1]]={u[0]+v[0], u[1]};
                pq.push({u[0]+v[0], u[1], v[1]});
            }
        }
    }
}

int main() {
    ifstream fin("dining.in");

```

```

    ofstream fout("dining.out");

    fin >> n >> m >> k;
    while(m--) {
        fin >> ai >> bi >> ti, --ai, --bi;
        adj[ai].push_back({ti, bi});
        adj[bi].push_back({ti, ai});
    }
    while(k--) {
        fin >> ai >> bi, --ai;
        y[ai]=max(bi, y[ai]);
    }
    dijkstra1();
    dijkstra2();

    for(int i=0; i<n-1; ++i)
        fout << (d2[i][1]<n-1||y[n-1]) << "\n";
}

```

2 transform

NOI2009-1 transform 变换序列

【问题描述】

对于 N 个整数 $0, 1, \dots, N-1$, 一个变换序列 T 可以将 i 变成 T_i , 其中 $T_i \in \{0, 1, \dots, N-1\}$ 且 $\bigcup_{i=0}^{N-1} T_i = \{0, 1, \dots, N-1\}$ 。 $\forall x, y \in \{0, 1, \dots, N-1\}$ 定义 x 和 y 之间的距离 $D(x, y) = \min\{|x - y|, N - |x - y|\}$ 。 给定每个 i 和 T_i 之间的距离 $D(i, T_i)$, 你要求出一个满足要求的变换序列 T 。 如果有多个满足条件的序列, 输出其中字典序最小的一个。

说明: 对于两个变换序列 S 和 T , 如果存在 $p < N$, 满足对于 $i = 0, 1, \dots, p-1$, $S_i = T_i$ 且 $S_p < T_p$, 我们称 S 比 T 字典序小。

【输入文件】

输入文件 *transform.in* 的第一行包含一个整数 N , 表示序列的长度。 接下来的一行包含 N 个整数 D_i , 其中 D_i 表示 i 和 T_i 之间的距离。

【输出文件】

输出文件为 *transform.out*。

如果至少存在一个满足要求的变换序列 T , 则输出文件中包含一行 N 个整数, 表示你计算得到的字典序最小的 T ; 否则输出 "NoAnswer" (不含引号)。 注意: 输出文件中相邻两个数之

间用一个空格分开，行末不包含多余空格。

【输入样例】

5

1 1 2 2 1

【输出样例】

1 2 4 0 3

【数据规模和约定】

20% 的数据中 $N \leq 50$;

60% 的数据中 $N \leq 500$;

100% 的数据中 $N \leq 10000$ 。

解:

【问题简述】

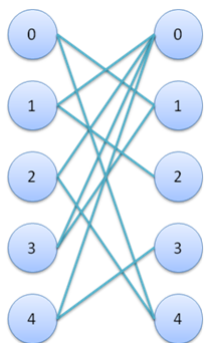
N 个整数 $0, 1, \dots, N-1$

input: D 距离序列 $D_0, D_1, \dots, D_{N-1}$, 包含 N 个整数 D_i 。

output: T 变换序列 $T_0, T_1, \dots, T_{N-1}$, 使得每个 i, T_i 的环上距离等于 D_i 。

一个合法的变换序列应是 $0, 1, \dots, N-1$ 的一个排列，任务是要求出字典序最小的那个变换序列。

【问题建模】 数学建模：抽象成图论模型，建立一个二分图，原始序列 X 集合每个顶点代表 $0, 1, \dots, N-1$ 的 N 个整数， Y 集合每个顶点为对应的 N 个整数。 X 集合的第 i 个顶点向其环上距离为 D_i 的 Y 集合中的两个顶点连一条边。样例建图后如图 1 所示。



建立一个二分图, X 集合每个顶点代表 $0, 1, \dots, N-1$ 的 N 个整数, Y 集合每个顶点为对应的 N 个整数。 X 集合的第 i 个顶点向其环上距离为 D_i 的 Y 集合中的两个顶点连一条边。

一个合法的变换序列应是 $0, 1, \dots, N-1$ 的一个排列。

【算法描述】

按照解法 3 证明中的描述，我们可以预处理所有已经确定的匹配，并在图中删去。对于剩下的每个环，只需从序号最小的点开始深度优先搜索，并进行匹配即可。

【算法证明】

【证明同解法 3】

【复杂度分析】

预处理的时间复杂度为 $O(N)$ ，深度优先搜索的时间复杂度为 $O(N)$ ，所以总时间复杂度为

$O(N)$ 。

```
#include <iostream>
#include <fstream>
#include <cstdlib>
#include <cmath>
#include <cstring>

using namespace std;

const int maxN = 20001, maxM = maxN * 4;

struct edge {
    edge* next;
    int t;
} *v[maxN], e[maxM];

int n;
int ec;
int mat[maxN];
int stack[maxN];
int s[maxN][2];
int deg[maxN];
bool nuked[maxN];

inline void addEdge(int a, int b) {
    e[++ec].next = v[a];
    v[a] = e + ec;
    v[a] -> t = b;

    e[++ec].next = v[b];
    v[b] = e + ec;
    v[b] -> t = a;

    ++deg[b];
    mat[b] = a;
}
```

```
void init() {
    ifstream fin("transform.in");
    fin >> n;

    int i, d, t1, t2;

    for (i = 1; i <= n; ++i) {
        fin >> d;

        t1 = i + d;
        if (t1 > n){
            t1 -= n;
        }

        t2 = i - d;
        if (t2 < 1){
            t2 += n;
        }

        if (t1 < t2){
            s[i][0] = t1;
            s[i][1] = t2;
        }else{
            s[i][0] = t2;
            s[i][1] = t1;
        }

        addEdge(i, s[i][1] += n);
        addEdge(i, s[i][0] += n);
        deg[i] = 2;
    }

    fin.close();
}

void noAnswer() {
    printf("No□Answer\n");
}
```

```

        exit(0);
    }

    inline void match(int i, int j) {
        mat[i] = j;
        mat[j] = i;
    }

    void prune() {
        int stop = 0, i, j;

        for (i = 1; i <= n; i++) {
            if (deg[i + n] == 0) {
                noAnswer();
            } else if (deg[i + n] == 1) {
                stack[++stop] = i + n;
            }
        }

        while(stop) {
            i = stack[stop--];
            nuked[i] = true;

            for (edge *e = v[i]; e; e = e->next) {
                j = e->t;
                if (!nuked[j]) {
                    break;
                }
            }

            match(i, j);
            i = j;
            nuked[i] = true;

            for (edge *e = v[i]; e; e = e->next) {
                j = e->t;
                if (!nuked[j]) {

```

```

                                deg[j]--;
                                if(deg[j] == 0){
                                    noAnswer();
                                }else if(deg[j] == 1){
                                    stack[++stop] = j;
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

void dfs(int i, bool s) {
    nuked[i] = true;
    int j;

    for (edge *e=v[i]; e; e=e->next) {
        j = e->t;
        if(!nuked[j]){
            dfs(j, !s);
            if(s){
                match(i, j);
            }
            break;
        }
    }
}

```

```

void solve() {
    prune();
    for (int i=1; i <= n; i++) {
        if (!nuked[i]) {
            dfs(i, true);
        }
    }
}

```

```

void print() {

```

```
    ofstream fout("transform.out");

    int i;
    for(i = 1; i < n; i++){
        fout << mat[i] - n - 1 << "□";
    }

    fout << mat[i] - n - 1 << "\\n";
    fout.close();
}

int main() {
    init();
    solve();
    print();
    return 0;
}
```

3 power

power-csp-j-2020

【题目描述】

一般来说，一个正整数可以拆分成若干个正整数的和。例如， $1 = 1$ $10 = 1 + 2 + 3 + 4$ 等。对于正整数 n 的一种特定拆分，我们称它为“优秀的”，当且仅当在这种拆分下 n 被分解为了若干个不同的 2 的正整数次幂。注意，一个数 x 能被表示成 2 的正整数次幂，当且仅当 x 能通过正整数个 2 相乘在一起得到。

例如， $10 = 8 + 2 = 2^3 + 2^1$ 是一个优秀的拆分。但是， $7 = 4 + 2 + 1 = 2^2 + 2^1 + 2^0$ 就不是一个优秀的拆分，因为 1 不是 2 的正整数次幂。

现在，给定正整数 n ，你需要判断这个数的所有拆分中，是否存在优秀的拆分。若存在，请你给出具体的拆分方案。

【输入格式】

输入文件名为 power.in。

输入文件只有一行，一个正整数 n ，代表需要判断的数。

【输出格式】

输出文件名为 power.out。

如果这个数的所有拆分中，存在优秀的拆分。那么，你需要从大到小输出这个拆分中的每一

个数，相邻两个数之间用一个空格隔开。可以证明，在规定了拆分数字的顺序后，该拆分方案是唯一的。

若不存在优秀的拆分，输出“-1”（不包含双引号）。

【样例 1 输入】

6

【样例 1 输出】

4 2

【样例 1 解释】

$6 = 4 + 2 = 2^2 + 2^1$ 是一个优秀的拆分。注意， $6 = 2 + 2 + 2$ 不是一个优秀的拆分，因为拆分成的 3 个数不满足每个数互不相同。

【样例 2 输入】

7

【样例 2 输出】

-1

【样例 3】

见选手目录下的 power/power3.in 与 power/power3.ans。

【数据范围与提示】

对于 20% 的数据， $n \leq 10$ 。

对于另外 20% 的数据，保证 n 为奇数。

对于另外 20% 的数据，保证 n 为 2 的正整数次幂。

对于 80% 的数据， $n \leq 1024$ 。

对于 100% 的数据， $1 \leq n \leq 1 \times 10^7$ 。

解：

1	input	6	7
2	output	4 2	-1
3	$2^1, 2^2, \dots, 2^n$	$power = 1 \ll i$	

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
    ifstream fin("power.in");
    ofstream fout("power.out");
    int n;
    vector<int> partition;
```

```

    fin >> n;
    for(int i = 30; i >= 0; i--){
        int power = 1<<i;
        if(n >= power){
            partition.push_back(power);
            n -= power;
        }
    }
    for(int i = 0; i <= partition.size(); i++){
        if(partition[i] != 0){
            fout<<partition[i];
            fout<<" ";
        }
    }
    fin.close();
    fout.close();
    return 0;
}

```

4 expr

expr-csp-j-2022-3

【题目描述】

逻辑表达式是计算机科学中的重要概念和工具，包含逻辑值、逻辑运算、逻辑运算优先级等内容。

在一个逻辑表达式中，元素的值只有两种可能：0（表示假）和1（表示真）。元素之间有多种可能的逻辑运算，本题中只需考虑如下两种：“与”（符号为 &）和“或”（符号为 |）。其运算规则如下：

$0 \& 0 = 0 \& 1 = 1 \& 0 = 0$, $1 \& 1 = 1$;

$0 | 0 = 0$, $0 | 1 = 1 | 0 = 1 | 1 = 1$ 。

在一个逻辑表达式中还可能有关括号。规定在运算时，括号内的部分先运算；两种运算并列时，& 运算优先于 | 运算；同种运算并列时，从左向右运算。

比如，表达式 $0 | 1 \& 0$ 的运算顺序等同于 $0 | (1 \& 0)$ ；表达式 $0 \& 1 \& 0 | 1$ 的运算顺序等同于 $((0 \& 1) \& 0) | 1$ 。

此外，在 C++ 等语言的有些编译器中，对逻辑表达式的计算会采用一种“短路”的策略。：在形如 $a \& b$ 的逻辑表达式中，会先计算 a 部分的值，如果 $a = 0$ ，那么整个逻辑表达式的值就一定为 0，故无需再计算 b 部分的值；同理，在形如 $a | b$ 的逻辑表达式中，会先计算 a 部

分的值，如果 $a = 1$ ，那么整个逻辑表达式的值就一定为 1，无需再计算 b 部分的值。

现在给你一个逻辑表达式，你需要计算出它的值，并且统计出在计算过程中，两种类型的“短路”各出现了多少次。需要注意的是，如果某处“短路”包含在更外层被“短路”的部分内则不被统计，如表达式 $1|(0\&1)$ 中，尽管 $0\&1$ 是一处“短路”，但由于外层的 $1|(0\&1)$ 本身就是一处“短路”，无需再计算 $0\&1$ 部分的值，因此不应当把这里的 $0\&1$ 计入一处“短路”。

【输入格式】

从文件 `expr.in` 中读入数据。

输入共一行，一个非空字符串 s 表示待计算的逻辑表达式。

【输出格式】

输出到文件 `expr.out` 中。

输出共两行，第一行输出一个字符 0 或 1，表示这个逻辑表达式的值；第二行输出两个非负整数，分别表示计算上述逻辑表达式的过程中，形如 $a/\&b$ 和 $a|b$ 的“短路”各出现了多少次。

【样例 1 输入】

$0\&(1|0)|(1|1|1\&0)$

【样例 1 输出】

1

1 2

【样例 1 解释】

该逻辑表达式的计算过程如下，每一行的注释表示上一行计算的过程：

$0\&(1|0)|(1|1|1\&0)$

$= (0\&(1|0)) | ((1|1) | (1\&0))$ //用括号标明计算顺序

$= 0 | ((1|1) | (1\&0))$ //先计算最左侧的 $\&$ ，是一次形如 $a\&b$ 的“短路”

$= 0 | (1 | (1\&0))$ //再计算中间的 $|$ ，是一次形如 $a|b$ 的“短路”

$= 0 | 1$ //再计算中间的 $|$ ，是一次形如 $a|b$ 的“短路”

$= 1$

【样例 2 输入】

$(0|1\&0|1|1|(1|1))\&(0\&1\&(1|0)|0|1|0)\&0$

【样例 2 输出】

0

2 3

```
#include <iostream>
#include <fstream>
#include <stack>
#include <cstring>
using namespace std;
```



```
struct node {
    int v;
    int y;
    int h;
};

int main() {
    ifstream fin("expr.in");
    ofstream fout("expr.out");

    char s[1000002];
    stack<char> q;
    stack<node> n;

    fin >> s;
    int l = strlen(s);
    s[l] = ')';
    q.push('(');

    for (int i = 0; i <= l; i++) {
        if (s[i] == '(') {
            q.push(s[i]);
        }
        else if (s[i] == '0' || s[i] == '1') {
            node temp;
            temp.v = s[i] - '0';
            temp.y = 0;
            temp.h = 0;
            n.push(temp);
        }
        else {
            bool fff = false;
            if (s[i] == ')') {
                fff = true;
                if (q.top() == '(') {
                    q.pop();
                }
            }
        }
    }
}
```

```

    else {
    while (!n.empty() && !q.empty() &&
(q.top() == '&' || q.top() == '|')) {
        char z = q.top();
        q.pop();
        node a = n.top();
        n.pop();
        node b = n.top();
        n.pop();
        node temp;
        temp.y = b.y;
        temp.h = b.h;
        if (z == '&') {
            if (b.v == 0) temp.y++;
            else {
                temp.y += a.y;
                temp.h += a.h;
            }
            temp.v = a.v & b.v;
        }
        if (z == '|') {
            if (b.v == 1) temp.h++;
            else {
                temp.h += a.h;
                temp.y += a.y;
            }
            temp.v = a.v | b.v;
        }
        n.push(temp);
    }
    q.pop();
}

if (!n.empty() && !q.empty()) {
while ((q.top() == '&' && (s[i] == '&' || s[i] == '|')) ||
(q.top() == '|' && s[i] == '|')) {
    char z = q.top();

```

```

        q.pop();
        node a = n.top();
        n.pop();
        node b = n.top();
        n.pop();
        node temp;
        temp.y = b.y;
        temp.h = b.h;
        if (z == '&') {
            if (b.v == 0) temp.y++;
            else {
                temp.y += a.y;
                temp.h += a.h;
            }
            temp.v = a.v & b.v;
        }
        if (z == '|') {
            if (b.v == 1) temp.h++;
            else {
                temp.h += a.h;
                temp.y += a.y;
            }
            temp.v = a.v | b.v;
        }
        n.push(temp);
    }
}

if (fff == false)
    q.push(s[i]);
}

fout << n.top().v << endl;
fout << n.top().y << '␣' << n.top().h << endl;

fin.close();
fout.close();

```

```
    return 0;  
}
```

5 pow

pow-csp-j-2022-1

【题目描述】

小文同学刚刚接触了信息学竞赛，有一天她遇到了这样一个题：给定正整数 a 和 b ，求 a^b 的值是多少。

a^b 即 b 个 a 相乘的值，例如 2^3 即为 3 个 2 相乘，结果为 $2 \times 2 \times 2 = 8$ 。

“简单！”小文心想，同时很快就写出了一份程序，可是测试时却出现了错误。

小文很快意识到，她的程序里的变量都是 `int` 类型的。在大多数机器上，`int` 类型能表示的最大数为 $2^{31}-1$ ，因此只要计算结果超过这个数，她的程序就会出现错误。

由于小文刚刚学会编程，她担心使用 `int` 计算会出现问题。因此她希望你在 a^b 的值超过 10^9 时，输出一个 -1 进行警示，否则就输出正确的 a^b 的值。

然而小文还是不知道怎么实现这份程序，因此她想请你帮忙。

【输入格式】

从文件 `pow.in` 中读入数据。

输入共一行，两个正整数 a, b 。

【输出格式】

输出到文件 `pow.out` 中。

输出共一行，如果 a^b 的值不超过 10^9 ，则输出 a^b 的值，否则输出 -1。

【样例 1 输入】

10 9

【样例 1 输出】

1000000000

【样例 2 输入】

23333 66666

【样例 2 输出】

-1

【数据范围】

对于 10% 的数据，保证 $b = 1$

对于 30% 的数据，保证 $b \leq 2$

对于 60% 的数据，保证 $b \leq 30, a^b \leq 10^{18}$

对于 100% 的数据，保证 $1 \leq a, b \leq 10^9$

解:

1	result *= a;
---	--------------

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

```
int main(){
    ifstream fin("pow.in");
    ofstream fout("pow.out");

    int a,b;
    fin>>a>>b;
    long long result = 1;
    for(int i = 1; i <= b; i++){
        result *= a;
        if(result > 1e9){
            fout<<-1;
            return 0;
        }
    }
    fout<<result;
    return 0;
}
```

6 number

number-csp-j-2019-1

【题目描述】

小 K 同学向小 P 同学发送了一个长度为 8 的 01 字符串来玩数字游戏，小 P 同学想要知道字符串中究竟有多少个 1。

注意：01 字符串为每一个字符是 0 或者 1 的字符串，如 “101”（不含双引号）为一个长度为 3 的 01 字符串。

【输入格式】

输入文件名为 number.in

输入文件只有一行，一个长度为 8 的 01 字符串 s 。

【输出格式】

输出文件名为 number.out

输出文件只有一行，包含一个整数，即 01 字符串中字符 1 的个数。

【输入输出样例 1】

number.in	number.out
00010100	2

【输入输出样例 2】

number.in	number.out
11111111	8

解:

1	s[i] == '1'
2	count++;

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main(){
    ifstream fin("number.in");
    ofstream fout("number.out");

    string s;
    fin>>s;
    int count = 0;

    for(int i = 0; i < s.length(); i++){
        if(s[i] == '1'){
            count++;
        }
    }
    fout<<count;

    fin.close();
    fout.close();
    return 0;
}
```

7 decode

decode-csp-j-2022-2

【题目描述】

给定一个正整数 k ，有 k 次询问，每次给定三个正整数 n_i, e_i, d_i 求两个正整数 p_i, q_i ，使 $n_i = p_i \times q_i, e_i \times d_i = (p_i - 1)(q_i - 1) + 1$ 。

【输入格式】

从文件 decode.in 中读入数据。

第一行一个正整数 k ，表示有 k 次询问。

接下来 k 行，第 i 行三个正整数 n_i, d_i, e_i 。

【输出格式】

输出到文件 decode.out 中。

输出 k 行，每行两个正整数 p_i, q_i

表示答案。

为使输出统一，你应当保证 $p_i \leq q_i$ 。

如果无解，请输出 NO。

【样例 1 输入】

1	10
2	770 77 5
3	633 1 211
4	545 1 499
5	683 3 227
6	858 3 257
7	723 37 13
8	572 26 11
9	867 17 17
10	829 3 263
11	528 4 109

【样例 1 输出】

1	2 385
2	NO
3	NO
4	NO
5	11 78
6	3 241
7	2 286
8	NO
9	NO
10	6 88

【样例 2】

见选手目录下的 decode/decode2.in 与 decode/decode2.ans。

【样例 3】

见选手目录下的 decode/decode3.in 与 decode/decode3.ans。

【样例 4】

见选手目录下的 decode/decode4.in 与 decode/decode4.ans。

【数据范围】

以下记 $m = n - e \times d + 2$ 。

保证对于 100% 的数据, $1 \leq k \leq 10^5$, 对于任意的 $1 \leq i \leq k, 1 \leq n_i \leq 10^{18}, 1 \leq e_i \times d_i \leq 10^{18}, 1 \leq m \leq 10^9$ 。

测试点编号	$k \leq$	$n \leq$	$m \leq$	特殊性质
1	10^3	10^3	10^3	保证有解
2				无
3				保证有解
4		10^9	6×10^4	无
5				保证有解
6				无
7	10^5	10^{18}	10^9	保证若有解则 $p = q$
8				保证有解
9				无
10				

解:

1	input n_i, e_i, d_i	770 77 5
2	output p_i, q_i	2 385
3	$n_i = p_i \times q_i$	$770 = 2 \times 385$
4	$e_i \times d_i = (p_i - 1)(q_i - 1) + 1$	$77 \times 5 = (2 - 1) \times (385 - 1) + 1$

$$\therefore e_i \times d_i = (p_i - 1)(q_i - 1) + 1$$

$$e_i d_i = p_i q_i - (p_i + q_i) + 2$$

$$\therefore p_i + q_i = p_i q_i - e_i d_i + 2$$

$$\text{令 } s = p_i + q_i$$

$$\text{则有 } s = p_i q_i - e_i d_i + 2$$

$$s = n - d * e + 2$$

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

```
int main(){
    ifstream fin("decode.in");
    ofstream fout("decode.out");
    int k;
    fin>>k;
    for(int i = 0;i < k;i++){
        long long n,d,e;
        fin>>n>>d>>e;
        long long s = n - d * e + 2;
        long long left = 1,right = s - 1;
        long long p = -1, q = -1;

        while(left <= right){
            long long mid = left + (right - left) / 2;
            if(mid * (s - mid) == n){
                p = mid;
                q = s - mid;
                break;
            }else if(mid*(s-mid)<n){
                left = mid + 1;
            }else{
                right = mid - 1;
            }
        }
        if(p != -1 && q != -1 && p <= q){
            fout<<p<<"□"<<q<<endl;
        }else{
            fout<<"NO"<<endl;
        }
    }
}
```

```

        }
    }
    fin.close();
    fout.close();
    return 0;
}

```

8 two-sum

<https://leetcode.com/problems/two-sum/>

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

```

#include <iostream>
#include <vector>
#include <unordered_map>

using namespace std;

vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> numIndexMap;

    for(int i = 0; i < nums.size(); i++){
        int complement = target - nums[i];
        if(numIndexMap.find(complement) != numIndexMap.end()){
            return {numIndexMap[complement], i};
        }
        numIndexMap[nums[i]];
    }
    return {};
}

```

```
int main() {
    vector<int> nums = {2, 7, 11, 15};
    int target = 9;
    vector<int> result = twoSum(nums, target);
    if (!result.empty()) {
        cout << result[0] << " " << result[1] << endl;
    } else {
        cout << "NO" << endl;
    }
    return 0;
}
```

9 candy

candy-csp-j-2021-2

【题目背景】

红太阳幼儿园的小朋友们开始分糖果啦！

【题目描述】

红太阳幼儿园有 n 个小朋友，你是其中之一。保证 $n \geq 2$ 。

有一天你在幼儿园的后花园里发现无穷多颗糖果，你打算拿一些糖果回去分给幼儿园的小朋友们。

由于你只是个平平无奇的幼儿园小朋友，所以你的体力有限，至多只能拿 R 块糖回去。

但是拿的太少不够分的，所以你至少要拿 L 块糖回去。保证 $n \leq L \leq R$ 。

也就是说，如果你拿了 k 块糖，那么你需要保证 $L \leq k \leq R$ 。

如果你拿了 k 块糖，你将把这 k 块糖放到篮子里，并要求大家按照如下方案分糖果：只要篮子里有不少于 n 块糖果，幼儿园的所有 n 个小朋友（包括你自己）都从篮子中拿走恰好一块糖，直到篮子里的糖数量少于 n 块。此时篮子里剩余的糖果均归你所有——这些糖果是作为你搬糖果的奖励。

作为幼儿园高质量小朋友，你希望让作为你搬糖果的奖励的糖果数量（而不是你最后获得的总糖果数量！）尽可能多；因此你需要写一个程序，依次输入 n, L, R ，并输出你最多能获得多少作为你搬糖果的奖励的糖果数量。

【输入格式】

从文件 candy.in 中读入数据。

输入一行，包含三个正整数 n, L, R ，分别表示小朋友的个数、糖果数量的下界和上界。

【输出格式】

输出到文件 candy.out 中。

输出一行一个整数，表示你最多能获得的作为你搬糖果的奖励的糖果数量。

【样例 1 输入】

7 16 23

【样例 1 输出】

6

【样例 1 解释】

拿 $k = 20$ 块糖放入篮子里。

篮子里现在糖果数 $20 \geq n = 7$ ，因此所有小朋友获得一块糖；

篮子里现在糖果数变成 $13 \geq n = 7$ ，因此所有小朋友获得一块糖；

篮子里现在糖果数变成 $6 < n = 7$ ，因此这 6 块糖是作为你搬糖果的奖励。容易发现，你获得的作为你搬糖果的奖励的糖果数量不可能超过 6 块（不然，篮子里的糖果数量最后仍然不少于 n ，需要继续每个小朋友拿一块），因此答案是 6。

【样例 2 输入】

10 14 18

【样例 2 输出】

8

【样例 2 解释】

容易发现，当你拿的糖数量 k 满足 $14 = L \leq k \leq R = 18$ 时，所有小朋友获得一块糖后，剩下的 $k - 10$ 块糖总是作为你搬糖果的奖励的糖果数量，因此拿 $k = 18$ 块是最优解，答案是 8。

【数据范围】

测试点	$n \leq$	$R \leq$	$R - L \leq$
1	2	5	5
2	5	10	10
3	10^3	10^3	10^3
4	10^5	10^5	10^5
5	10^3 10^5 10^9	10^9	0
6			10^3
7			10^5
8			10^9
9			
10			

对于所有数据，保证 $2 \leq n \leq L \leq R \leq 10^9$

解:

1	input	7 16 23	n, L, R	小朋友的个数、糖果数量的下界和上界
2	output	6	k	你最多能获得的糖果数量

```
#include <iostream>
#include <fstream>
```

```
using namespace std;

int main(){
    ifstream fin("candy.in");
    ofstream fout("candy.out");
    int n,l,r;
    int k;
    fin>>n>>l>>r;
    if(l<n*(r/n)&& n*(r/n)<r){
        k = n * (r/n) -1-n*(r/n-1);
    }else if(n*(r/n)<l){
        k = r - n;
    }
    fout<<k;
    return 0;
}
```

10 live

live-csp-j-2020-2

直播获奖 (live)

【题目描述】

NOI2130 即将举行。为了增加观赏性,CCF 决定逐一评出每个选手的成绩,并直播即时的获奖分数线。本次竞赛的获奖率为 $w\%$,即当前排名前 $w\%$ 的选手的最低成绩就是即时的分数线。

更具体地,若当前已评出了 p 个选手的成绩,则当前计划获奖人数为 $\max(1, \lfloor p \times w\% \rfloor)$,其中 w 是获奖百分比, $\lfloor x \rfloor$ 表示对 x 向下取整, $\max(x, y)$ 表示 x 和 y 中较大的数。如有选手成绩相同,则所有成绩并列的选手都能获奖,因此实际获奖人数可能比计划中多。

作为评测组的技术人员,请你帮 CCF 写一个直播程序。

【输入格式】

输入文件名为 live.in。

第 1 行两个正整数 n, w 。分别代表选手总数与获奖率。

第 2 行有 n 个非负整数,依次代表逐一评出的选手成绩。

【输出格式】

输出文件名为 live.out。

只有一行，包含 n 个非负整数，依次代表选手成绩逐一评出后，即时的获奖分数线。相邻两个整数间用一个空格分隔。

【样例 1 输入】

```
10 60
200 300 400 500 600 600 0 300 200 100
```

【样例 1 输出】

```
200 300 400 400 400 500 400 400 300 300
```

解:

```
#include <iostream>
#include <fstream>

using namespace std;

int n,w,x;
int a[610];

int main(){
    ifstream fin("live.in");
    ofstream fout("live.out");
    fin>>n>>w;

    int k;
    int ans;
    for(int i = 1; i <= n; i++){
        fin>>x;
        a[x]++;
        k = max(1,int(i * w / 100.0));
        ans = 0;
        for(int j = 600;j >= 0; j--){
            if(ans + a[j] < k){
                ans += a[j];
            } else {
                fout<<j<<" ";
                break;
            }
        }
    }
}
```

```
fout << "\n";

fin.close();
fout.close();
return 0;
}
```

11 number

number-csp-j-2020-4

方格取数 number

【题目描述】

设有 $n \times m$ 的方格图，每个方格中都有一个整数。现有一只小熊，想从图的左上角走到右下角，每一步只能向上、向下或向右走一格，并且不能重复经过已经走过的方格，也不能走出边界。小熊会取走所有经过的方格中的整数，求它能取到的整数之和的最大值。

【输入格式】

输入文件名为 number.in。

第 1 行两个正整数 n, m 。

接下来 n 行每行 m 个整数，依次代表每个方格中的整数。

【输出格式】

输入文件名为 number.out。

一个整数，表示小熊能取到的整数之和的最大值。

【样例 1 输入】

```
3 4
1 -1 3 2
2 -1 4 -1
-2 2 -3 -1
```

【样例 1 输出】

9

【样例 1 解释】

1	-1	3	2
2	-1	4	-1
-2	2	-3	-1

按上述走法，取到的数之和为 $1 + 2 + (-1) + 4 + 3 + 2 + (-1) + (-1) = 9$ ，可以证明为最大

值。

解:

```
#include <iostream>
#include <fstream>

using namespace std;

int n,m;
const long long minNum = -1e18;
long long gridValue[1005][1005];
long long memorizationTable[1005][1005][2];

inline long long maxNum(long long p, long long q, long long r){
    return p > q ? (p > r ? p : r) : (q > r ? q : r);
}

long long dfs(int x,int y,int from){
    if(x < 1 && x > n || y < 1 && y > m){
        return minNum;
    }

    if(memorizationTable[x][y][from] != minNum){
        return memorizationTable[x][y][from];
    }

    if(from == 0){
        memorizationTable[x][y][from] = maxNum(dfs(x + 1, y
            , 0),dfs(x, y - 1, 0),dfs(x, y - 1, 1)) +
            gridValue[x][y];
    }else{
        memorizationTable[x][y][from] = maxNum(dfs(x - 1, y
            , 1),dfs(x, y - 1, 0),dfs(x, y - 1, 1)) +
            gridValue[x][y];
    }

    return memorizationTable[x][y][from];
}
```



```

int main(){
    ifstream fin("number.in");
    ofstream fout("number.out");

    fin>>n>>m;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            fin>>gridValue[i][j];
            memorizationTable[i][j][0] =
                memorizationTable[i][j][1] = minNum;
        }
    }
    memorizationTable[1][1][0] = memorizationTable[1][1][1] =
        minNum;

    fout<<dfs(n,m,1)<<endl;
    fin.close();
    fout.close();
    return 0;
}

```

12 expr

expr-csp-j-2020-3

表达式 (expr)

【题目描述】

小 C 热衷于学习数理逻辑。有一天，他发现了一种特别的逻辑表达式。在这种逻辑表达式中，所有操作数都是变量，且它们的取值只能为 0 或 1，运算从左往右进行。如果表达式中有括号，则先计算括号内的子表达式的值。特别的，这种表达式有且仅有以下几种运算：

1. 与运算： $a \& b$ 。当且仅当 a 和 b 的值都为 1 时，该表达式的值为 1。其余情况该表达式的值为 0。
2. 或运算： $a | b$ 。当且仅当 a 和 b 的值都为 0 时，该表达式的值为 0。其余情况该表达式的值为 1。
3. 取反运算： $!a$ 。当且仅当 a 的值为 0 时，该表达式的值为 1。其余情况该表达式的值为 0。

小 C 想知道，给定一个逻辑表达式和其中每一个操作数的初始取值后，再取反某一个操作数

的值时，原表达式的值为多少。

为了化简对表达式的处理，我们有如下约定：

表达式将采用后缀表达式的方式输入。后缀表达式的定义如下：

1. 如果 E 是一个操作数，则 E 的后缀表达式是它本身。
2. 如果 E 是 $E_1 \text{ op } E_2$ 形式的表达式，其中 op 是任何二元操作符，且优先级不高于 E_1, E_2 中括号外的操作符，则 E 的后缀式为 $E_1' E_2' \text{op}$ ，其中 E_1', E_2' 分别为 E_1, E_2 的后缀式。
3. 如果 E 是 (E_1) 形式的表达式，则 (E_1) 的后缀式就是 E 的后缀式。

同时为了方便，输入中：

- a) 与运算符 ($\&$)、或运算符 ($|$)、取反运算符 ($!$) 的左右均有一个空格，但表达式末尾没有空格。
- b) 操作数由小写字母 x 与一个正整数拼接而成，正整数表示这个变量的下标。例如： x_{10} ，表示下标为 10 的变量 x_{10} 。数据保证每个变量在表达式中出现恰好一次。

【输入格式】

输入文件名为 `expr.in`。

第一行包含一个字符串 s ，表示上文描述的表达式。

第二行包含一个正整数 n ，表示表达式中变量的数量。表达式中变量的下标为 $1, 2, \dots, n$ 。

第三行包含 n 个整数，第 i 个整数表示变量 x_i 的初值。

第四行包含一个正整数 q ，表示询问的个数。

接下来 q 行，每行一个正整数，表示需要取反的变量的下标。注意，每一个询问的修改都是临时的，即之前询问中的修改不会对后续的询问造成影响。数据保证输入的表达式合法。变量的初值为 0 或 1。

【输出格式】

输出文件名为 `expr.out`。

输出一共有 行，每行一个 0 或 1，表示该询问下表达式的值。

【样例 1 输入】

`x1 x2 & x3 |`

`3`

`1 0 1`

`3`

`1`

`2`

`3`

【样例 1 输出】

`1 1 0`

【样例 1 解释】

该后缀表达式的中缀表达式形式为 $(1\&2)|3$ 。

对于第一次询问，将 1 的值取反。此时，三个操作数对应的赋值依次为 0, 0, 1。原表达式

的值为 $(0 \& 0) | 1 = 1$ 。

对于第二次询问，将 2 的值取反。此时，三个操作数对应的赋值依次为 1, 1, 1。原表达式的值为 $(1 \& 1) | 1 = 1$ 。

对于第三次询问，将 3 的值取反。此时，三个操作数对应的赋值依次为 1, 0, 0。原表达式的值为 $(1 \& 0) | 0 = 0$ 。

解:

【条件分析】

条件编号	条件描述
1	表达式中的操作数只能取值为 0 或 1
2	运算从左往右进行
3	如果表达式中有括号，则先计算括号内的子表达式的值
4	表达式中只包含三种运算符：与运算 (&)、或运算 (
5	与运算 (&) 的结果为 1 的条件是，两个操作数的值都为 1；否则结果为 0
6	或运算 (
7	取反运算 (!) 的结果为 1 的条件是，操作数的值为 0；否则结果为 0
8	表达式采用后缀表达式的方式输入，即运算符在操作数之后
9	后缀表达式的定义：

13 point

point-csp-2022-4

【题目描述】

在一个二维平面内，给定 n 个整数点 (x_i, y_i) ，此外你还可以自由添加 k 个整数点。你在自由添加 k 个点后，还需要从 $n + k$ 个点中选出若干个整数点并组成一个序列，使得序列中任意相邻两点间的欧几里得距离恰好为 1 而且横坐标、纵坐标值均单调不减，即 $x_{i+1} - x_i = 1$, $y_{i+1} = y_i$ 或 $y_{i+1} - y_i = 1$, $x_{i+1} = x_i$ 。请给出满足条件的序列的最大长度。

【输入格式】

从文件 point.in 中读入数据。

第一行两个正整数 n, k 分别表示给定的整点个数、可自由添加的整点个数。接下来 n 行，第 i 行两个正整数 x_i, y_i 表示给定的第 i 个点的横纵坐标。

【输出格式】

输出到文件 point.out 中。

输出一个整数表示满足要求的序列的最大长度。

【样例 1 输入】

8 2
3 1
3 2
3 3
3 6
1 2
2 2
5 5
5 3

【样例 1 输出】

8

【样例 2 输入】

4 100
10 10
15 25
20 20
30 30

【样例 2 输出】

103

解:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
```

```
using namespace std;
```

```
int main(){
    ifstream fin("point.in");
    ofstream fout("point.out");

    int n,k;
    fin>>n>>k;
    int ans = 0;

    vector<pair<int,int>> points(n);
    for(int i = 0; i < n; i++){
```

```

        int x,y;
        fin>>x>>y;
        points[i] = {x,y};
    }

    sort(points.begin(),points.end());

    vector<vector<int>> dp(n,vector<int>(k + 1,0));
    for(int i = 0; i < n; i++){
        for(int j = 0; j <= k; j++){
            dp[i][j] = j + 1;
            for(int t = 0; t < i; t++){
                pair<int,int> &A = points[i];
                pair<int,int> &B = points[t];
                int d = A.first - B.first + A.
                    second - B.second - 1;
                if(A.second >= B.second){
                    if(j >= d){
                        dp[i][j] = max(dp[i]
                            ][j],dp[t][j - d
                                ] + d + 1);
                    }
                }
            }
            ans = max(ans,dp[i][j]);
        }
    }

    fout<<ans<<endl;
    fin.close();
    fout.close();
    return 0;
}

```

14 fruit

fruit-csp-2021-4

小熊的果篮 (fruit)

【题目描述】

小熊的水果店里摆放着一排 n 个水果。每个水果只可能是苹果或桔子，从左到右依次用正整数 1、2、3、……、 n 编号。连续排在一起的同一种水果称为一个“块”。小熊要把这一排水果挑到若干个果篮里，具体方法是：每次都把每一个“块”中最左边的水果同时挑出，组成一个果篮。重复这一操作，直至水果用完。注意，每次挑完一个果篮后，“块”可能会发生变化。比如两个苹果“块”之间的唯一桔子被挑走后，两个苹果“块”就变成了一个“块”。请帮小熊计算每个果篮里包含的水果。

【输入格式】

从文件 fruit.in 中读入数据。

输入的第一行包含一个正整数 n ，表示水果的数量。

输入的第二行包含 n 个空格分隔的整数，其中第 i 个数表示编号为 i 的水果的种类，1 代表苹果，0 代表桔子。

【输出格式】

输出到文件 fruit.out 中。

输出若干行。

第 i 行表示第 i 次挑出的水果组成的果篮。从小到大排序输出该果篮中所有水果的编号，每两个编号之间用一个空格分隔。

【样例 1 输入】

12
1 1 0 0 1 1 1 0 1 1 0 0

【样例 1 输出】

1 3 5 8 9 11
2 4 6 12
7
10

【样例 1 解释】

这是第一组数据的样例说明。

所有水果一开始的情况是 1 1 0 0 1 1 1 0 1 1 0 0 一共有 6 个块。

在第一次挑水果组成果篮的过程中，编号为 1 3 5 8 9 11 的水果被挑了出来。

之后剩下的水果是 1 0 1 1 1 0，一共 4 个块。

在第二次挑水果组成果篮的过程中，编号为 2 4 6 12 的水果被挑了出来。

之后剩下的水果是 1 1，只有 1 个块。

在第三次挑水果组成果篮的过程中，编号为 7 的水果被挑了出来。

最后剩下的水果是 1，只有 1 个块。

在第四次挑水果组成果篮的过程中，编号为 10 的水果被挑了出来。

解：

```
#include <bits/stdc++.h>

#define INF 200010

using namespace std;

int n;
set<int> s1, s2; // 两个集合，分别存储元素的下标

int main() {
    ifstream fin("fruit.in"); // 从文件 "fruit.in" 读取输入
    ofstream fout("fruit.out"); // 输出结果到文件 "fruit.out"

    fin >> n; // 读取元素数量 n
    s1.clear(); // 清空集合 s1
    s2.clear(); // 清空集合 s2
    int q;
    for (int i = 1; i <= n; ++i) {
        fin >> q; // 读取输入元素 q
        if (q) s1.insert(i); // 若 q 非零，将当前下标 i 插入集合 s1
        else s2.insert(i); // 若 q 为零，将当前下标 i 插入集合
            s2
    }
    s1.insert(INF); // 在集合 s1 中插入 INF，用于防止集合为空
    s2.insert(INF); // 在集合 s2 中插入 INF，用于防止集合为空
    int nw = 0;
    bool p = *s1.begin() < *s2.begin() ? 0 : 1; // 根据两个集合的最
        小值确定 p 值，表示当前删的是哪个集合

    while (!p && s1.size() > 1 || p && s2.size() > 1) {
        if (!p) {
            nw = *s1.upper_bound(nw); // 找到集合 s1 中下一个大于
                nw 的元素
        }
    }
}
```

```

        if (nw == INF) {
            nw = 0;
            p = *s1.begin() < *s2.begin() ? 0 : 1; // 若找不到
                下一个元素, 重新确定 p 值
            fout << endl; // 换行
            continue;
        }
        fout << nw << '␣'; // 输出 nw 并删除它
        s1.erase(nw);
        p = !p; // 切换到另一个集合
    } else {
        nw = *s2.upper_bound(nw);
        if (nw == INF) {
            nw = 0;
            p = *s1.begin() < *s2.begin() ? 0 : 1;
            fout << endl;
            continue;
        }
        fout << nw << '␣';
        s2.erase(nw);
        p = !p;
    }
}

fout << ""<<endl; // 输出最后一个果篮并换行
while (s1.size() > 1) fout << *s1.begin() << endl, s1.erase(*s1
    .begin()); // 逐行输出 s1 剩余元素
while (s2.size() > 1) fout << *s2.begin() << endl, s2.erase(*s2
    .begin()); // 逐行输出 s2 剩余元素

return 0;
}

```

15 sort

sort-csp-2021-2

【题目描述】

插入排序是一种非常常见且简单的排序算法。小 Z 是一名大一的新生，今天 H 老师刚刚在上课的时候讲了插入排序算法。

假设比较两个元素的时间为 $O(1)$ ，则插入排序可以以 $O(n^2)$ 的时间复杂度完成长度为 n 的数组的排序。不妨假设这 n 个数字分别存储在 $a_1, a_2, \dots, a_n$ 之中，则如下伪代码给出了插入排序算法的一种最简单的实现方式：

这下面是 C/C++ 的示范代码

```
for(int i = 1; i <= n; i++){
    for(int j = i; j >= 2; j--){
        if(a[j] < a[j - 1]){
            int t = a[j - 1];
            a[j - 1] = a[j];
            a[j] = t;
        }
    }
}
```

为了帮助小 Z 更好的理解插入排序，小 Z 的老师 H 老师留下了这么一道家庭作业：

H 老师给了一个长度为 n 的数组 a ，数组下标从 1 开始，并且数组中的所有元素均为非负整数。小 Z 需要支持在数组 a 上的 Q 次操作，操作共两种，参数分别如下：

1 $x \ v$ ：这是第一种操作，会将 a 的第 x 个元素，也就是 a_x 的值，修改为 v 。保证 $1 \leq x \leq n$ ， $1 \leq v \leq 10^9$ 。注意这种操作会改变数组的元素，修改得到的数组会被保留，也会影响后续的操作。

2 x ：这是第二种操作，假设 H 老师按照上面的伪代码对 a 数组进行排序，你需要告诉 H 老师原来 a 的第 x 个元素，也就是 a_x ，在排序后的新数组所处的位置。保证 $1 \leq x \leq n$ 。注意这种操作不会改变数组的元素，排序后的数组不会被保留，也不会影响后续的操作。

H 老师不喜欢过多的修改，所以他保证类型 1 的操作次数不超过 5000。

小 Z 没有学过计算机竞赛，因此小 Z 并不会做这道题。他找到了你来帮助他解决这个问题。

【输入格式】

从文件 sort.in 中读入数据。

输入的第一行包含两个正整数 n, q ，表示数组长度和操作次数。保证 $1 \leq n \leq 8,000, 1 \leq q \leq 2 \times 10^5$ 。

输入的第二行包含 n 个空格分隔的非负整数，其中第 i 个非负整数表示 a_i 。保证 $1 \leq a_i \leq 10^9$ 。接下来 q 行，每行 2 ~ 3 个正整数，表示一次操作，操作格式见题目描述。

【输出格式】

输出到文件 sort.out 中。

对于每一次类型为 2 的询问，输出一行一个正整数表示答案。

【样例 1 输入】

3 4
3 2 1
2 3
1 3 2
2 2
2 3

【样例 1 输出】

1
1
2

【样例 1 解释】

在修改操作之前，假设 H 老师进行了一次插入排序，则原序列的三个元素在排序结束后所处的位置分别是 3, 2, 1。

在修改操作之前，假设 H 老师进行了一次插入排序，则原序列的三个元素在排序结束后所处的位置分别是 3, 1, 2。

注意虽然此时 $a_2 = a_3$ ，但是我们不能将其视为相同的元素。

解：

```
#include <iostream>
#include <fstream>
#include <algorithm>
```

```
using namespace std;
```

```
struct node {
    int id;
    int num;
};
```

```
node a[8010];
int b[8010];
int n, q;
```

```
void change(int pos) {
    while (pos > 1) {
        if (a[pos].num == a[pos - 1].num && a[pos].id > a[pos - 1].
            id) break;
        if (a[pos].num > a[pos - 1].num) break;
```

```
        swap(a[pos], a[pos - 1]);
        b[a[pos].id] = pos;
        pos--;
    }
    while (pos < n) {
        if (a[pos].num == a[pos + 1].num && a[pos].id < a[pos + 1].
            id) break;
        if (a[pos].num < a[pos + 1].num) break;
        swap(a[pos], a[pos + 1]);
        b[a[pos].id] = pos;
        pos++;
    }
}

bool cmp(node a, node b) {
    if (a.num == b.num) return a.id < b.id;
    else return a.num < b.num;
}

int main() {
    ifstream fin("sort.in");
    ofstream fout("sort.out");

    fin >> n >> q;
    for (int i = 1; i <= n; i++) {
        fin >> a[i].num;
        a[i].id = i;
    }

    sort(a + 1, a + 1 + n, cmp);
    for (int i = 1; i <= n; i++) {
        b[a[i].id] = i;
    }

    while (q--) {
        int op, x, v;
        fin >> op;
```

```

        if (op == 1) {
            fin >> x >> v;
            a[b[x]].num = v;
            change(b[x]);
            for (int i = 1; i <= n; i++) {
                b[a[i].id] = i;
            }
        } else {
            fin >> x;
            fout << b[x] << endl;
        }
    }

    fin.close();
    fout.close();
    return 0;
}

```

16 solve

csp-j-2021-13

```

#include <iostream>

using namespace std;

int solve(int n){
    if(n <= 1){
        return 1;
    }else if(n >= 5){
        return n * solve(n-2); 2*solve(1)=2 solve(1)=1
    }else{
        return n*solve(n-1);
    }
    return 0;
}

```

```
int main() {  
    int i = 7;  
    int result = solve(7);  
    cout<<result<<endl;  
    return 0;  
}
```

17 network

network-csp-2021-3

【题目描述】

TCP/IP 协议是网络通信领域的一项重要协议。今天你的任务，就是尝试利用这个协议，还原一个简化后的网络连接场景。

在本问题中，计算机分为两大类：服务机（Server）和客户机（Client）。服务机负责建立连接，客户机负责加入连接。

需要进行网络连接的计算机共有 n 台，编号为 $1 \sim n$ ，这些机器将按编号递增的顺序，依次发起一条建立连接或加入连接的操作。

每台机器在尝试建立或加入连接时需要提供一个地址串。服务机提供的地址串表示它尝试建立连接的地址，客户机提供的地址串表示它尝试加入连接的地址。

一个符合规范的地址串应当具有以下特征：

- 1、必须形如 $a.b.c.d:e$ 的格式，其中 a, b, c, d, e 均为非负整数；
- 2、 $0 \leq a, b, c, d \leq 255, 0 \leq e \leq 65535$ ；
- 3、 a, b, c, d, e 均不能含有多余的前导 0。

相应地，不符合规范的地址串可能具有以下特征：

- 1、不是形如 $a.b.c.d:e$ 格式的字符串，例如含有多于 3 个字符. 或多于 1 个字符: 等情况；
- 2、整数 a, b, c, d, e 中某一个或多个超出上述范围；
- 3、整数 a, b, c, d, e 中某一个或多个含有多余的前导 0。例如，地址串 $192.168.0.255:80$ 是符合规范的，但 $192.168.0.999:80$ 、 $192.168.00.1:10$ 、 $192.168.0.1:088$ 、 $192:168:0:1.233$ 均是不符合规范的。如果服务机或客户机在发起操作时提供的地址串不符合规范，这条操作将被直接忽略。在本问题中，我们假定凡是符合上述规范的地址串均可参与正常的连接，你无需考虑每个地址串的实际意义。

由于网络阻塞等原因，不允许两台服务机使用相同的地址串，如果此类现象发生，后一台尝试建立连接的服务机将会无法成功建立连接；除此之外，凡是提供符合规范的地址串的服务机均可成功建立连接。

如果某台提供符合规范的地址的客户机在尝试加入连接时，与先前某台已经成功建立连接的

服务机提供的地址串相同，这台客户机就可以成功加入连接，并称其连接到这台服务机；如果找不到这样的服务机，则认为这台客户机无法成功加入连接。请注意，尽管不允许两台不同的服务机使用相同的地址串，但多台客户机使用同样的地址串，以及同一台服务机同时被多台客户机连接的情况是被允许的。你的任务很简单：在给出每台计算机的类型以及地址串之后，判断这台计算机的连接情况。

【输入格式】

从文件 `network.in` 中读入数据。

第 1 行，一个正整数 n 。

接下来 n 行，每行 2 个字符串 `op`, `ad`，按照编号从小到大给出每台计算机的类型及地址串。其中 `op` 保证为字符串 `Server` 或 `Client` 之一，`ad` 为一个长度不超过 25 的，仅由数字、字符和字符组成的非空字符串。

每行的两个字符串之间用恰好一个空格分隔开，每行的末尾没有多余的空格。

【输出格式】

输出到文件 `network.out` 中。

输出共 n 行，每行一个正整数或字符串表示第 i 台计算机的连接状态。其中：

如果第 i 台计算机为服务机，则：

1. 如果其提供符合规范的地址串且成功建立连接，输出字符串 `OK`。
2. 如果其提供符合规范的地址串，但由于先前有相同地址串的服务机而无法成功建立连接，输出字符串 `FAIL`。
3. 如果其提供的地址串不是符合规范的地址串，输出字符串 `ERR`。

如果第 i 台计算机为客户机，则：

1. 如果其提供符合规范的地址串且成功加入连接，输出一个正整数表示这台客户机连接到的服务机的编号。
2. 如果其提供符合规范的地址串，但无法成功加入连接时，输出字符串 `FAIL`。
3. 如果其提供的地址串不是符合规范的地址串，输出字符串 `ERR`。

【样例 1 输入】

```
5
192.168.1.1:8080
Server 192.168.1.1:8080
Client 192.168.1.1:8080
Client 192.168.1.1:80
Client 192.168.1.1:99999
```

【样例 1 输出】

```
OK
FAIL
1
FAIL
ERR
```

解：

18 josephus

josephus-csp-2021

(Josephus 问题) 有 n 个人围成一个圈, 依次标号 0 至 $n - 1$ 。

从 0 号开始, 依次 $0, 1, 0, 1, \dots$ 交替报数, 报到 1 的人会离开, 直至圈中只剩下一个人。

求最后剩下人的编号。

求解方法:

解决 Josephus 问题的经典方法是使用递归或数学公式。

可以得到一个递推关系式, 用 $f(n, m)$ 表示 n 个人报数, 报到 m 时最后剩下的人的编号。

递推关系式为:

$$f(n, m) = (f(n - 1, m) + m) \% n$$

其中, $f(1, m) = 0$, 表示只剩下一个人时的情况。

这个关系式的意义是: 在 n 个人报数的情况下,

第一轮报数时, 第一个被淘汰的人的编号为 $(f(n-1, m) + m)$ 然后问题转化为 $n-1$ 个人报数的情况, 并从下一个人开始重新报数。

通过不断递归计算 $f(n, m)$, 最终可以得到只剩下最后一个人的编号, 即 Josephus 问题的解。

解:

```
#include <iostream>
```

```
using namespace std;
```

```
const int MAXN = 1000000; // 最大的人数
```

```
int F[MAXN]; // 标记每个人是否被淘汰出圈
```

```
int main() {
```

```
    int n;
```

```
    cin >> n; // 接收输入的人数  $n$ 
```

```
    int i = 0, p = 0, c = 0; // 当前报数的人编号、报数的标记、以及已经被淘汰的人数
```

```
    while(c < n - 1) { // 进入一个 while 循环, 循环条件是还未剩下最后一个人
```

```
        if(F[i] == 0) { // 判断当前报数的人是否还在圈内, 如果  $F[i]$  为  $0$ , 表示该人还在圈内
```

```

        if(p){ // 判断当前报数标记p的值。p的初始值为
            0, 当p为1时, 说明报数到1i, 该人将被淘汰
            F[i] = 1; // 将当前报数的人标记为1
                , 表示被淘汰
            c++; // 然后c增加1, 表示已经淘汰的
                人数加1
        }
        p ^= 1; // 将p的值进行异或运算, 相当于p取反,
            即0变成1, 1变成0。这样可以实现交替报数的
            效果
    }
    i = (i + 1) % n; // 更新报数的人的编号i, 取余操作确
        保编号在圈内循环
}
int ans = -1; // 定义一个变量ans, 用于记录最后剩下的人的编
    号, 初始化为-1
for(i = 0; i < n; i++) // 遍历数组F, 查找最后剩下的人的编号
    if(F[i] == 0) // 如果F[i]为0, 表示该人未被淘汰
        ans = i; // 将该人的编号赋值给ans
cout << ans << endl;
return 0;
}

```

19 convention

usaco-2018-convention

Farmer John is hosting a new bovine grass-eating convention at his farm! Cows from all over the world are arriving at the local airport to attend the convention and eat grass. Specifically, there are N cows arriving at the airport ($1 \leq N \leq 10^5$) and cow i arrives at time t_i ($0 \leq t_i \leq 10^9$). Farmer John has arranged M ($1 \leq M \leq 10^5$) buses to transport the cows from the airport. Each bus can hold up to C cows in it ($1 \leq C \leq N$). Farmer John is waiting with the buses at the airport and would like to assign the arriving cows to the buses. A bus can leave at the time when the last cow on it arrives. Farmer John wants to be a good host and so does not want to keep the arriving cows waiting at the airport too long. What is the smallest possible value of the maximum waiting time of any one arriving cow if Farmer John coordinates his buses optimally? A cow's waiting time is the difference between her arrival time and the departure

of her assigned bus.

It is guaranteed that $MC \geq N$.

INPUT FORMAT (file convention.in):

The first line contains three space separated integers N , M , and C . The next line contains N space separated integers representing the arrival time of each cow.

OUTPUT FORMAT (file convention.out):

Please write one line containing the optimal minimum maximum waiting time for any one arriving cow.

SAMPLE INPUT:

6 3 2

1 1 10 14 4 3

SAMPLE OUTPUT:

4

If the two cows arriving at time 1 go in one bus, cows arriving at times 3 and 4 in the second, and cows arriving at times 10 and 14 in the third, the longest time a cow has to wait is 4 time units (the cow arriving at time 10 waits from time 10 to time 14).

解:

【条件分析】

1	N 头牛抵达机场	$1 \leq N \leq 10^5$
2	第 i 头牛在时间 t_i 抵达	$0 \leq t_i \leq 10^9$
3	M 辆巴士来运牛从机场到农场	$1 \leq M \leq 10^5$
4	每辆巴士最多可以容纳 C 头牛	$1 \leq C \leq N$
4	input	6 3 2
4	output	4